
Phân công code vấn đáp đồ án

X509Certificate

Phần việc chi tiết của thành viên

Nguyễn Gia Huy

*Nền tảng hệ thống, Bảo mật dữ liệu
và Khung giao diện*

Tài liệu này mô tả chi tiết phần code, lý thuyết bảo mật cần nắm, các test case và cách tích hợp cho cụm việc **nền tảng hệ thống (bootstrap, CSDL), bảo mật dữ liệu (hashing & encryption-at-rest) và khung giao diện (design system, router, các dialog dùng chung)**, gắn trực tiếp với mã nguồn thực tế của đồ án: `main.py`, `src/config.py`, `src/db/`, `src/core/encryption.py`, `src/services/auth.py`, `src/services/audit.py`, `src/ui/` và `scripts/backup_db.py`.

Môn: Mã hoá và Ứng dụng (MHUD)

Học phần đồ án X.509 Certificate

Ngày biên soạn: 14/06/2026

Contents

1	Mục tiêu và phạm vi phần việc	2
1.1	Ảnh xạ khái niệm → file code → vai trò	2
2	Kiến trúc tổng thể và vị trí của thành viên	4
2.1	Luồng khởi động (bootstrap)	4
2.2	Phân tầng kiến trúc	5
3	Phần code cần làm (chi tiết)	5
3.1	Điểm vào và neo thư mục làm việc — <code>main.py</code>	5
3.2	Hàng số tập trung — <code>config.py</code>	6
3.3	Lớp kết nối SQLite — <code>db/connection.py</code>	6
3.4	Lược đồ cơ sở dữ liệu — <code>db/schema.sql</code>	8
3.5	Bảo mật dữ liệu — <code>core/encryption.py</code>	9
3.5.1	Băm mật khẩu bằng scrypt (một chiều)	10
3.5.2	Mã hóa private key bằng AES-256-GCM (hai chiều)	10
3.6	Xác thực người dùng — <code>services/auth.py</code>	11
3.7	Nhật ký hệ thống — <code>services/audit.py</code>	12
3.8	Khung ứng dụng & router — <code>ui/app.py</code>	13
3.9	Hệ thống thiết kế (design system) — <code>ui/theme.py</code>	14
3.10	Đọc & hiển thị chi tiết certificate — <code>CertDetailDialog</code>	15
3.11	Màn hình đăng nhập & widget tái dụng	16
3.12	Sao lưu dữ liệu — <code>scripts/backup_db.py</code>	16
4	Cần nắm để vấn đáp	17
5	Test case & demo	19
5.1	Bảng test case chính	19
5.2	Đoạn unit test gợi ý (tự kiểm chứng nhanh khi demo)	19
5.3	Kịch bản demo trực quan	20
6	Tích hợp vào chương trình chung	20
7	Bổ sung sau rà soát: các feature dễ bị hỏi	21
7.1	Wiring chế độ LAN trong <code>app.py</code> & <code>login.py</code>	21
7.2	Quy tắc hợp lệ username & mật khẩu (<code>auth.py</code>)	22
7.3	Server hạ tầng nền & Backup/Restore đầy đủ	22
7.4	Chi tiết nhỏ khác	22
8	Checklist chuẩn bị vấn đáp	23

1 Mục tiêu và phạm vi phân việc

Theo bảng phân công chung, thành viên **Nguyễn Gia Huy** phụ trách cụm **nền tảng hệ thống, bảo mật dữ liệu và khung giao diện**. Đây là phần *đặt nền móng* cho toàn bộ đồ án: nếu các thành viên khác viết nghiệp vụ X.509 (sinh khóa, ký CSR, kiểm tra chuỗi chứng chỉ, CRL & OCSP) thì cụm của Huy cung cấp *hạ tầng* để mọi nghiệp vụ đó chạy được:

- **Khởi động & cấu hình**: điểm vào `main.py`, neo thư mục làm việc, và file hằng số tập trung `config.py`.
- **Tầng dữ liệu (CSDL)**: lớp kết nối SQLite (`db/connection.py`) với WAL, `foreign_keys`, quản lý transaction; và lược đồ 9 bảng (`db/schema.sql`).
- **Bảo mật dữ liệu lúc nghỉ (at-rest)**: `core/encryption.py` — băm mật khẩu bằng script (một chiều) và mã hóa private key bằng AES-256-GCM (hai chiều).
- **Xác thực & nhật ký**: `services/auth.py` (đăng ký, đăng nhập, đổi mật khẩu, chống dò user) và `services/audit.py` (ghi nhật ký best-effort).
- **Khung giao diện (GUI shell)**: `ui/app.py` (router single-root), `ui/theme.py` (design system), `ui/common.py` (các dialog dùng chung — trong đó có hộp thoại *đọc & hiển thị chi tiết certificate*), `ui/login.py`, các widget tái dụng và vỏ dashboard.
- **Vận hành (ops)**: tiện ích sao lưu `scripts/backup_db.py` với SQLite backup API và manifest băm SHA-256.

1.1 Ánh xạ khái niệm → file code → vai trò

Bảng dưới đây là tấm bản đồ tổng để định vị nhanh từng khái niệm trong mã nguồn. Hội đồng vấn đáp thường yêu cầu “chỉ ra trong code chỗ làm việc X” — bảng này giúp trả lời ngay.

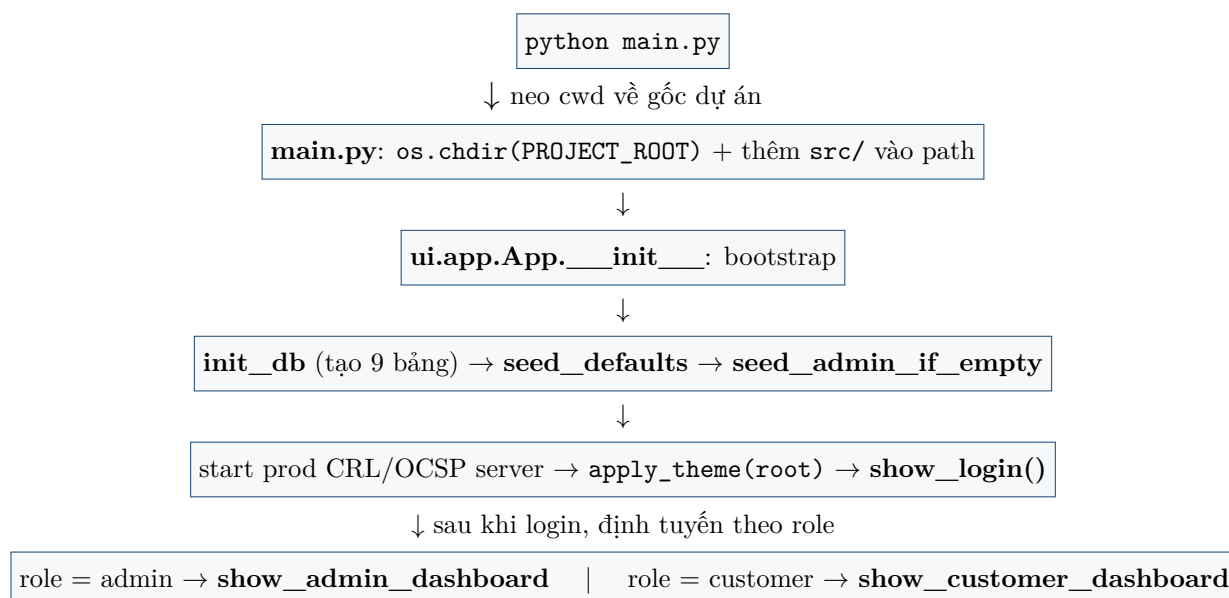
Khái niệm	File code	Vai trò
Điểm vào & neo cwd	<code>main.py</code>	<code>os.chdir(PROJECT_ROOT)</code> để mọi relative path (DB, master.key, certs/) trở về gốc dự án; gọi <code>ui.app.main</code> .
Hằng số tập trung	<code>src/config.py</code>	Gom path, validity mặc định, giới hạn HTTP về 1 nơi — “1 chỗ đổi, không grep”.
Kết nối SQLite	<code>src/db/connection.py</code>	<code>get_conn</code> (WAL + foreign_keys ON + Row factory), <code>init_db</code> , context manager <code>transaction</code> / <code>conn_scope</code> .
Lược đồ CSDL	<code>src/db/schema.sql</code>	9 bảng, khóa ngoại, ràng buộc CHECK, index cho truy vấn nóng.
Băm mật khẩu (1 chiều)	<code>src/core/encryption.py</code>	<code>hash_password</code> / <code>verify_password</code> bằng scrypt + salt ngẫu nhiên.
Mã hóa dữ liệu (2 chiều)	<code>src/core/encryption.py</code>	<code>encrypt_blob</code> / <code>decrypt_blob</code> bằng AES-256-GCM + master.key + nonce + AAD.
Xác thực người dùng	<code>src/services/auth.py</code>	<code>register_user</code> , <code>login</code> , <code>change_password</code> ; chống dò user qua dummy verify.
Nhật ký hệ thống	<code>src/services/audit.py</code>	<code>write_audit</code> (best-effort) + catalog Action; <code>list_recent</code> .
Vỏ ứng dụng (router)	<code>src/ui/app.py</code>	1 Tk root, <code>show_*</code> , bootstrap DB/seed/server, định tuyến theo role.
Design system	<code>src/ui/theme.py</code>	Token màu, spacing, font Montserrat, <code>apply_theme</code> cấu hình ttk Style.
Dialog dùng chung	<code>src/ui/common.py</code>	<code>CertDetailDialog</code> (decode & hiển thị X.509), <code>ChangePasswordDialog</code> , header dashboard.
Màn hình đăng nhập	<code>src/ui/login.py</code>	Form login/register + panel LAN.
Widget tái dụng	<code>src/ui/widgets/</code>	<code>init_modal</code> / <code>fit_to_content</code> ; <code>StatusFilterTreeFrame</code> .
Vỏ dashboard	<code>src/ui/admin/</code> , <code>src/ui/customer/</code>	Sidebar menu + content router theo nhóm chức năng.
Sao lưu dữ liệu	<code>scripts/backup_db.py</code>	SQLite backup API + manifest SHA-256 cho yêu cầu nộp D.2.

Định vị nhanh: cụm của Huy phủ ba tầng dưới cùng của kiến trúc: *config* → *data* → *security* → *UI shell*. Mọi nghiệp vụ X.509 của các bạn khác đều “ngồi lên” tầng này: gọi `get_conn/transaction` để đọc ghi DB, gọi `encrypt_blob` để cất private key, gọi `write_audit` để ghi log, và render giao diện trong khung App đã thiết lập sẵn.

2 Kiến trúc tổng thể và vị trí của thành viên

Ứng dụng là một desktop app viết bằng Python + Tkinter, dữ liệu lưu trong SQLite, khóa bí mật được mã hóa at-rest. Sơ đồ dưới đây mô tả luồng khởi động và các tầng; phần **in đậm** là cụm việc của Huy.

2.1 Luồng khởi động (bootstrap)



2.2 Phân tầng kiến trúc

Tầng	Thành phần (cụm của Huy in đậm)
Trình bày (UI)	app.py , theme.py , common.py , login.py , widgets/ , vỏ dashboard; (các view nghiệp vụ do thành viên khác viết, render bên trong khung này).
Dịch vụ (service)	auth.py , audit.py , <code>system_config</code> , <code>infra_manager</code> , <code>ca</code> , <code>csr</code> , <code>cert_lifecycle</code> ...
Lỗi mật mã (core)	encryption.py (cụm Huy), <code>verify.py</code> , <code>keygen</code> ...
Dữ liệu (data)	connection.py , schema.sql (SQLite).
Cấu hình & ops	config.py , scripts/backup_db.py .

Tư duy “ai gọi ai”: cụm của Huy hầu như *không gọi* lên các tầng trên, mà *được gọi* từ các tầng trên. Đây là kiểu phụ thuộc một chiều (dependency hướng xuống) giúp tầng nền ổn định, dễ test độc lập.

3 Phần code cần làm (chi tiết)

3.1 Điểm vào và neo thư mục làm việc — `main.py`

`main.py` làm hai việc quan trọng trước khi mở giao diện: (1) *neo* thư mục làm việc về gốc dự án để mọi đường dẫn tương đối luôn ổn định, (2) thêm `src/` vào `sys.path` rồi gọi `ui.app.main`.

```

1 PROJECT_ROOT = Path(__file__).resolve().parent
2
3 # Anchor cwd về project root → mọi relative path trong app (master.key,
4 # ca_app.db, certs/, received_certs/, backups/...) đều resolve về đây
5 # thay vì cwd nơi user chạy command.
6 os.chdir(PROJECT_ROOT)
7 sys.path.insert(0, str(PROJECT_ROOT / "src"))
8
9 from ui.app import main
10
11 if __name__ == "__main__":
12     main()
```

Giải thích từng ý.

- `Path(__file__).resolve().parent` cho đường dẫn tuyệt đối của thư mục chứa `main.py` — chính là gốc dự án.

- `os.chdir(PROJECT_ROOT)` đổi thư mục làm việc hiện tại (`cwd`). Vì app dùng các path tương đối như `"ca_app.db"`, `"master.key"`, `"certs/"`, nếu không neo `cwd` thì khi chạy từ thư mục khác các file này sẽ bị tạo *nhầm chỗ*.
- `sys.path.insert(0, .../src)` cho phép import gọn (`from ui.app import main`) thay vì phải dùng package path dài.

3.2 Hằng số tập trung — `config.py`

File này gom các “magic value” về một nơi để tránh sửa rải rác. Nguyên tắc: path để *tương đối*, các giá trị validity chỉ là *mặc định UI* (runtime ưu tiên đọc từ bảng `system_config`).

```

1 # Production data
2 CERTS_DIR      = "certs"
3 TRUST_STORE_CERT = "certs/trust_store/root_ca.crt"
4 # Runtime app data (root project)
5 DB_FILE        = "ca_app.db"
6 MASTER_KEY_FILE = "master.key"
7 # Cert lifecycle defaults (UI fallback)
8 DEFAULT_VALIDITY_DAYS      = 365
9 DEFAULT_ROOT_CA_VALIDITY_DAYS = 3650
10 DEFAULT_KEY_SIZE           = 2048
11 # HTTP fetch limits
12 MAX_CRL_BYTES      = 10 * 1024 * 1024
13 MAX_OCSP_BYTES     = 64 * 1024

```

3.3 Lớp kết nối SQLite — `db/connection.py`

Đây là điểm *duy nhất* mở kết nối DB trong toàn app. Mỗi connection được cấu hình ba thứ then chốt: bật khóa ngoại, đặt Row factory, và bật WAL.

```

1 def get_conn(db_path: str = DEFAULT_DB_PATH) -> sqlite3.Connection:
2     """Mở SQLite connection ở chế độ autocommit + foreign_keys ON."""
3     conn = sqlite3.connect(db_path, isolation_level=None)
4     conn.row_factory = sqlite3.Row
5     conn.execute("PRAGMA foreign_keys = ON")
6     # WAL mode - tốt cho desktop app với reader đồng thời (UI + nền).
7     conn.execute("PRAGMA journal_mode = WAL")
8     return conn

```

Giải thích.

- `isolation_level=None`: tắt cơ chế tự mở transaction ngầm của `sqlite3` — ta chủ động `BEGIN/COMMIT` (xem context manager bên dưới). Như vậy biên giới transaction là *tường minh*.

- `row_factory = sqlite3.Row`: cho phép truy cập cột bằng tên (`row["username"]`) thay vì chỉ số — code dễ đọc, ít lỗi.
- `PRAGMA foreign_keys = ON`: SQLite mặc định **TẮT** khóa ngoại; phải bật cho *mỗi connection* thì các ràng buộc REFERENCES trong schema mới có hiệu lực.
- `PRAGMA journal_mode = WAL` (Write-Ahead Logging): cho phép đọc và ghi đồng thời tốt hơn — phù hợp app có nhiều luồng đọc (UI + server nền).

Hai context manager quản lý vòng đời connection một cách an toàn:

```

1  @contextmanager
2  def transaction(db_path: str = DEFAULT_DB_PATH):
3      conn = get_conn(db_path)
4      try:
5          conn.execute("BEGIN")
6          try:
7              yield conn
8          except Exception:
9              conn.execute("ROLLBACK")    # có lỗi → hủy toàn bộ
10             raise
11         else:
12             conn.execute("COMMIT")      # không lỗi → ghi nhận
13     finally:
14         conn.close()
15
16  @contextmanager
17  def conn_scope(db_path: str = DEFAULT_DB_PATH):
18      """Read-only - chỉ mở/đóng conn, không bao BEGIN/COMMIT."""
19      conn = get_conn(db_path)
20      try:
21          yield conn
22     finally:
23         conn.close()

```

- `transaction`: dùng cho thao tác *ghi*. Mọi câu lệnh trong khối `with` hoặc *thành công cùng nhau* (COMMIT) hoặc *thất bại cùng nhau* (ROLLBACK) — đảm bảo tính nguyên tử (atomicity).
- `conn_scope`: dùng cho thao tác *đọc* — gọn hơn, không tốn chi phí BEGIN/COMMIT.

`init_db` đọc `schema.sql` và chạy một lần; nhờ `CREATE TABLE IF NOT EXISTS` nên gọi lại nhiều lần cũng an toàn (idempotent):

```

1  def init_db(db_path: str = DEFAULT_DB_PATH) -> None:
2      schema = SCHEMA_PATH.read_text(encoding="utf-8")
3      conn = get_conn(db_path)
4      try:
5          conn.executescript(schema)    # idempotent: IF NOT EXISTS

```



```

6     finally:
7         conn.close()

```

3.4 Lược đồ cơ sở dữ liệu — db/schema.sql

Schema gồm **9 bảng** liên kết bằng khóa ngoại. Quy ước chung: thời gian là TEXT (ISO-8601 UTC), `password_hash` là chuỗi tự-mô-tả (không mã hóa), còn `encrypted_private_key` + `gcm_nonce` là dữ liệu AES-256-GCM.

Bảng	Vai trò
<code>users</code>	Tài khoản (admin/customer); CHECK role; <code>password_hash</code> .
<code>system_config</code>	Cặp key-value cấu hình hệ thống.
<code>root_ca</code>	Root CA (CN, serial, <code>cert_pem</code> , private key mã hóa).
<code>customer_keys</code>	Keypair khách hàng; private key mã hóa + nonce.
<code>csr_requests</code>	Yêu cầu cấp cert; trạng thái pending/approved/rejected.
<code>issued_certs</code>	Cert đã phát hành; có <code>renewed_from_id</code> , <code>revoked_at</code> .
<code>revocation_requests</code>	Yêu cầu thu hồi cert.
<code>audit_log</code>	Nhật ký hoạt động (actor, action, target, details, time).
<code>external_certs</code>	Cert ngoài do customer upload để theo dõi/verify.

Ví dụ một bảng có nhiều ràng buộc tiêu biểu — bảng `users`:

```

1 CREATE TABLE IF NOT EXISTS users (
2     id            INTEGER PRIMARY KEY AUTOINCREMENT,
3     username      TEXT      NOT NULL UNIQUE,
4     password_hash TEXT      NOT NULL,
5     role          TEXT      NOT NULL CHECK(role IN ('admin', 'customer')),
6     created_at    TEXT      NOT NULL,
7     last_login_at TEXT
8 );

```

Và một bảng dùng khóa ngoại + chính sách xóa (cascade / set null / restrict) — trích bảng `csr_requests`:

```

1 CREATE TABLE IF NOT EXISTS csr_requests (
2     id            INTEGER PRIMARY KEY AUTOINCREMENT,
3     requester_id  INTEGER NOT NULL REFERENCES users(id) ON DELETE CASCADE,
4     customer_key_id INTEGER NOT NULL REFERENCES customer_keys(id)
5                                     ON DELETE RESTRICT,
6     common_name   TEXT      NOT NULL,
7     status        TEXT      NOT NULL DEFAULT 'pending'

```

```

8             CHECK(status IN ('pending','approved','rejected')),
9     submitted_at    TEXT    NOT NULL,
10    reviewed_by     INTEGER REFERENCES users(id) ON DELETE SET NULL
11 );

```

Ý nghĩa các chính sách khóa ngoại.

- ON DELETE CASCADE: xóa user thì các CSR của user đó cũng bị xóa theo — tránh dữ liệu mồ côi.
- ON DELETE RESTRICT: *cấm* xóa keypair khi vẫn còn CSR tham chiếu tới nó — bảo vệ toàn vẹn.
- ON DELETE SET NULL: người duyệt bị xóa thì cột `reviewed_by` về NULL nhưng *giữ lại* bản ghi CSR.

Cuối schema là các index cho truy vấn nóng, ví dụ:

```

1 CREATE INDEX IF NOT EXISTS idx_audit_timestamp ON audit_log(timestamp DESC);
2 CREATE INDEX IF NOT EXISTS idx_issued_certs_revoked
3     ON issued_certs(revoked_at) WHERE revoked_at IS NOT NULL;
4 CREATE INDEX IF NOT EXISTS idx_csr_status ON csr_requests(status);

```

Index thứ hai là *partial index* (WHERE `revoked_at IS NOT NULL`) — chỉ đánh chỉ mục các cert đã thu hồi, vốn chiếm thiểu số. Nhỏ gọn mà vẫn tăng tốc truy vấn “liệt kê cert bị thu hồi” (phục vụ sinh CRL).

3.5 Bảo mật dữ liệu — `core/encryption.py`

Đây là phần lõi bảo mật và là điểm hỏi-đáp *trọng tâm* của cụm. Module có **hai cơ chế độc lập**, dùng cho hai mục đích khác nhau:

	Băm mật khẩu (<code>srypt</code>)	Mã hóa dữ liệu (AES-256-GCM)
Mục đích	Lưu mật khẩu để <i>kiểm chứng</i>	Cất private key để <i>dùng lại</i>
Chiều	Một chiều (không giải ngược)	Hai chiều (mã / giải mã)
Cần khóa?	Không (chỉ cần salt)	Có — <code>master.key</code> 32 byte
Đầu ra	<code>srypt\$N\$r\$p\$salt\$hash</code>	(nonce, ciphertext+tag)

3.5.1 Băm mật khẩu bằng scrypt (một chiều)

```

1 def hash_password(password: str) -> str:
2     salt = secrets.token_bytes(_SCRYPT_SALT_LEN)          # 16 byte ngẫu nhiên
3     derived = hashlib.scrypt(
4         password.encode("utf-8"),
5         salt=salt, n=_SCRYPT_N, r=_SCRYPT_R, p=_SCRYPT_P, # N=2^14, r=8, p=1
6         dklen=_SCRYPT_DKLEN,
7     )
8     return f"scrypt${_SCRYPT_N}${_SCRYPT_R}${_SCRYPT_P}${salt.hex()}${derived.hex()}"

1 def verify_password(password: str, hashed: str) -> bool:
2     # ... tách "scrypt$N$r$p$salt$hash" từ chuỗi đã lưu ...
3     derived = hashlib.scrypt(password.encode("utf-8"),
4                               salt=salt, n=n, r=r, p=p, dklen=len(expected))
5     return secrets.compare_digest(derived, expected) # so sánh hằng-thời-gian

```

Giải thích.

- **scrypt** là hàm dẫn xuất khóa *tốn bộ nhớ* (memory-hard): tham số $N = 2^{14}$, $r = 8$ làm mỗi lần băm tốn ≈ 64 MB và ≈ 0.1 s. Đủ chậm để chống brute-force offline nhưng vẫn mượt khi đăng nhập.
- **salt ngẫu nhiên 16 byte** mỗi mật khẩu: hai người dùng cùng mật khẩu sẽ cho hash khác nhau, vô hiệu hóa rainbow table.
- **Chuỗi tự-mô-tả** nhúng cả tham số N, r, p : sau này có thể *tăng* tham số cho mật khẩu mới mà vẫn xác thực được hash cũ.
- `secrets.compare_digest` so sánh *hằng thời gian* để tránh rò rỉ thông tin qua timing.

Vì sao không thể “giải ngược” mật khẩu? scrypt là hàm *một chiều*: từ mật khẩu suy ra hash thì dễ, nhưng từ hash suy ngược ra mật khẩu là bài toán không khả thi (phải dò vét). Vì vậy DB chỉ lưu hash — kể cả khi DB bị lộ, kẻ tấn công cũng không có mật khẩu gốc.

3.5.2 Mã hóa private key bằng AES-256-GCM (hai chiều)

```

1 def encrypt_blob(plaintext, aad=None, master_key_path=DEFAULT_MASTER_KEY_PATH):
2     nonce = secrets.token_bytes(GCM_NONCE_SIZE)          # 12 byte/lần
3     aesgcm = AESGCM(get_master_key(master_key_path))    # khóa 32 byte
4     ct = aesgcm.encrypt(nonce, plaintext, aad)           # tag gắn cuối ct
5     return nonce, ct
6
7 def decrypt_blob(nonce, ciphertext, aad=None, ...):
8     aesgcm = AESGCM(get_master_key(master_key_path))

```

```
9     return aesgcm.decrypt(nonce, ciphertext, aad) # sai key/nonce/aad → InvalidTag
```

Giải thích.

- **AES-256** dùng khóa 256-bit lấy từ `master.key` (32 byte ngẫu nhiên sinh lần đầu, sau đó đọc lại và cache).
- **GCM** (Galois/Counter Mode) là chế độ *mã hóa có xác thực* (AEAD): vừa giữ bí mật vừa phát hiện chỉnh sửa. Nếu ciphertext bị sửa một bit, `decrypt` sẽ ném `InvalidTag`.
- **nonce 12 byte** mới mỗi lần mã hóa — *tuyệt đối không tái dùng* nonce với cùng khóa (lặp nonce phá vỡ GCM). Vì vậy nonce được lưu kèm ciphertext trong DB.
- **AAD** (Associated Data) là dữ liệu *không mã hóa nhưng được xác thực*, dùng để “cột” ciphertext vào ngữ cảnh, ví dụ `f"customer_keys:{id}"`. Nếu kẻ tấn công đổi chỗ ciphertext của bản ghi này sang bản ghi khác, AAD sẽ không khớp ⇒ giải mã thất bại.

Master key được nạp lười và cache lại; lần đầu sẽ tự sinh và ghi file:

```
1 def get_master_key(path=DEFAULT_MASTER_KEY_PATH) -> bytes:
2     global _master_key_cache
3     if _master_key_cache is None:
4         _master_key_cache = _load_or_create_master_key(path) # sinh nếu chưa có
5     return _master_key_cache
```

Cảnh báo bảo mật (cần nói rõ khi vấn đáp): demo lưu `master.key` dạng file phẳng cạnh dự án để chạy offline. Trong môi trường thật, khóa này phải nằm trong HSM/KMS. Mất `master.key` = mất toàn bộ private key; lộ `master.key` = lộ toàn bộ private key.

3.6 Xác thực người dùng — `services/auth.py`

Module trả về dict (không bao giờ chứa `password_hash`) và ném `AuthError` khi nghiệp vụ sai. Điểm tinh tế nhất nằm ở login:

```
1 def login(username, password, db_path) -> dict:
2     row = conn.execute(
3         "SELECT id, username, password_hash, role FROM users WHERE username=?",
4         (username,)).fetchone()
5     # Vẫn gọi verify_password kể cả khi row=None để tránh user-enum qua timing.
6     pw_hash = row["password_hash"] if row else (
7         "scrypt$16384$8$1$" + "0" * 32 + "$" + "0" * 64) # hash giả
8     ok = verify_password(password, pw_hash)
9     if not row or not ok:
10         raise AuthError("Sai username hoặc password.") # message generic
```

```
11 # ... cập nhật last_login_at trong transaction nhỏ ...
```

Giải thích các quyết định thiết kế.

- **Chống dò tài khoản (user-enumeration):** nếu username không tồn tại, code *vẫn* chạy `verify_password` trên một hash giả. Nhờ đó thời gian phản hồi cho “sai user” và “sai password” gần như bằng nhau, kẻ tấn công không thể đoán username nào có thật qua đo thời gian.
- **Thông báo lỗi chung** (“Sai username hoặc password”) cũng nhằm không tiết lộ username có tồn tại hay không.
- Đăng ký (`register_user`) kiểm tra username trùng *bên trong* transaction để tránh race condition, bấm mật khẩu trước khi lưu.

Hàm `change_password` kiểm tra mật khẩu cũ, ép mật khẩu mới phải khác cũ, rồi bấm lại — tất cả trong một transaction:

```
1 def change_password(user_id, old_password, new_password, db_path) -> None:
2     _validate_password(new_password)
3     if old_password == new_password:
4         raise AuthError("Mật khẩu mới phải khác mật khẩu cũ.")
5     with transaction(db_path) as conn:
6         row = conn.execute("SELECT password_hash FROM users WHERE id=?",
7                             (user_id,)).fetchone()
8         if not verify_password(old_password, row["password_hash"]):
9             raise AuthError("Mật khẩu cũ không đúng.")
10        conn.execute("UPDATE users SET password_hash=? WHERE id=?",
11                      (hash_password(new_password), user_id))
```

3.7 Nhặt ký hệ thống — `services/audit.py`

Audit log ghi lại mọi hành động chính (login, đổi mật khẩu, duyệt CSR, thu hồi cert...).
Triết lý: **ghi log không bao giờ được phá vỡ nghiệp vụ.**

```
1 def write_audit(db_path, actor_id, action, *, target_type=None,
2               target_id=None, details=None) -> None:
3     now = datetime.now(timezone.utc).isoformat()
4     details_json = (json.dumps(details, ensure_ascii=False, default=str)
5                    if details else None)
6     try:
7         conn = get_conn(db_path)
8         try:
9             conn.execute("BEGIN")
10            conn.execute("INSERT INTO audit_log (actor_id, action, "
11                          "target_type, target_id, details_json, timestamp) "
12                          "VALUES (?, ?, ?, ?, ?, ?)",
```

```

13         (actor_id, action, target_type, target_id, details_json, now))
14         conn.execute("COMMIT")
15     finally:
16         conn.close()
17 except Exception as e:
18     print(f"[audit] FAILED to write event '{action}': {e}", file=sys.stderr)

```

Giải thích.

- **Best-effort:** toàn bộ thân hàm bọc trong `try/except`; nếu ghi log lỗi thì chỉ in ra `stderr`, *không* ném exception ra ngoài — nhờ vậy caller không cần `try/except` và nghiệp vụ chính không bị gián đoạn.
- **Catalog Action:** tập hằng các tên action (`LOGIN`, `CSR_APPROVED`, `CERT_REVOKED`...) định nghĩa tập trung để tránh gõ sai chuỗi rả rác.
- `details` là dict tùy ý được `json.dumps` với `ensure_ascii=False` (giữ tiếng Việt) — lưu vào cột `details_json`.

Giao diện xem log — `ui/admin/audit_view.py`. Bề mặt UI của audit là `AuditLogFrame`: bảng Treeview hiển thị 200 sự kiện mới nhất (`list_recent`), có ô lọc theo action và nút Lọc/Refresh. Đây là phần “đọc” tương ứng với phần “ghi” `write_audit` ở trên — cùng do Huy phụ trách, khép kín vòng nhật ký (ghi → xem).

3.8 Khung ứng dụng & router — `ui/app.py`

App là “bộ não” của giao diện: chỉ **một** Tk root, mỗi “trang” là một method `show_*` dọn root rồi render frame mới (giống mô hình SPA).

```

1 class App:
2     def __init__(self, db_path=DEFAULT_DB_PATH):
3         self.db_path = db_path
4         self.session = None
5         self.bootstrap()           # init_db + seed_defaults + seed_admin
6         self.infra = get_infra()
7         self.infra.start_prod_servers() # CRL + OCSP nền
8         self.root = tk.Tk()
9         apply_theme(self.root)       # design system (gọi sau khi có root)
10        self.show_login()
11
12    def on_login_success(self, user: dict) -> None:
13        self.session = user
14        write_audit(self.db_path, user["id"], Action.LOGIN, ...)
15        if user["role"] == "admin":
16            self.show_admin_dashboard() # định tuyến theo role
17        else:
18            self.show_customer_dashboard()

```

Giải thích.

- `_bootstrap` tạo schema, seed cấu hình mặc định, và seed admin mặc định lần đầu (`admin / Admin@123`) — in cảnh báo đỏ ngay.
- `apply_theme(self.root)` phải gọi *sau* khi tạo Tk root vì nó cấu hình `ttk.Style`.
- **Định tuyến theo role:** sau khi login thành công, dựa vào `user["role"]` để mở dashboard admin hay customer. `session` lưu `{id, username, role}` cho các trang dùng chung.
- `logout` ghi audit rồi quay về `show_login`.

3.9 Hệ thống thiết kế (design system) — `ui/theme.py`

`theme.py` định nghĩa *token* (màu, spacing, font) và hàm `apply_theme` cấu hình `ttk.Style` cho toàn app — đảm bảo giao diện nhất quán, đổi một chỗ áp dụng mọi nơi. Font chủ đạo là **Montserrat**.

```

1  COLOR = {
2      "primary": "#1E3A8A", "accent": "#10B981",
3      "bg": "#F8FAFC", "surface": "#FFFFFF", "border": "#E2E8F0",
4      "text": "#0F172A", "text_muted": "#475569",
5      "success": "#059669", "warning": "#D97706", "danger": "#DC2626",
6  }
7  SPACE = {"xs": 4, "sm": 8, "md": 12, "lg": 16, "xl": 24, "xxl": 32}
8  HEADING_FONT_CHAIN = ["Montserrat"]
9  BODY_FONT_CHAIN     = ["Montserrat"]

1 def apply_theme(root: tk.Tk) -> None:
2     root.configure(bg=COLOR["bg"])
3     style = ttk.Style(root)
4     style.theme_use("clam")           # base hỗ trợ custom màu tốt nhất
5     style.configure("Primary.TButton", background=COLOR["primary"],
6                     foreground=COLOR["text_inverse"], ...)
7     style.map("Primary.TButton",
8              background=[("active", COLOR["primary_hover"]),
7              ("pressed", COLOR["primary_active"])]])
9     # ... cấu hình TFrame, TLabel, TEntry, Treeview, Notebook ...
10

```

Giải thích.

- `font(role)` phân giải tên font theo “chain” (ưu tiên Montserrat, fallback an toàn), *cache* kết quả để không truy vấn lại danh sách font hệ thống.
- `style.map` định nghĩa trạng thái tương tác (hover/pressed/disabled) — ví dụ nút Primary đậm hơn khi nhấn.

- Vì Tkinter không có hiệu ứng kính (glassmorphism) thật, design system mô phỏng chiều sâu bằng *lớp bề mặt* (bg → surface → surface_alt) và viền tinh tế.

3.10 Đọc & hiển thị chi tiết certificate — CertDetailDialog

Đây là phần “*đọc/hiển thị cert*” thuộc cụm của Huy, dùng chung cho cả admin và customer. Dialog có hai tab: **Decoded** (giải mã các trường) và **PEM** (văn bản gốc + nút Copy/Save). Phần lõi là decode bằng thư viện cryptography:

```

1  from cryptography import x509
2  cert = x509.load_pem_x509_certificate(bytes(self.rec["cert_pem"]))
3  lines = [
4      f"Version          : {cert.version.name}",
5      f"Serial           : {cert.serial_number:#x}",
6      f"Signature Algo    : {cert.signature_algorithm_oid._name}",
7      f"Issuer           : {cert.issuer.rfc4514_string()}",
8      f"Subject          : {cert.subject.rfc4514_string()}",
9  ]
10 lines.append(f"Not Before      : {cert.not_valid_before_utc}")
11 lines.append(f"Not After       : {cert.not_valid_after_utc}")
12 pk = cert.public_key()
13 if hasattr(pk, "key_size"):
14     lines.append(f"Public Key      : {pk.__class__.__name__} {pk.key_size} bits")
15 lines.append("=== Extensions ===")
16 for ext in cert.extensions:
17     lines.append(f"    • {ext.oid._name} (critical={ext.critical})")
18     lines.append(f"        {ext.value}")

```

Các trường X.509 được trích và ý nghĩa.

- **Version**: phiên bản X.509 (thường là v3).
- **Serial**: số sê-ri duy nhất do CA cấp, in dạng hex.
- **Signature Algorithm**: thuật toán CA dùng để ký cert (vd sha256WithRSAEncryption).
- **Issuer / Subject**: tên phân biệt (DN) của *người cấp* và *chủ thể*, định dạng RFC 4514.
- **Not Before / Not After**: cửa sổ hiệu lực (validity window).
- **Public Key**: loại khóa và độ dài (vd RSA 2048 bits).
- **Extensions**: các phần mở rộng (BasicConstraints, KeyUsage, SubjectAltName, CRLDistributionPoints...), kèm cờ `critical`.

Toàn bộ thao tác decode bọc trong `try/except`; nếu PEM hỏng thì hiển thị “Không decode được cert PEM: ...” thay vì làm sập giao diện. Ô Text đặt `state=DISABLED` để chỉ-đọc.

3.11 Màn hình đăng nhập & widget tái dụng

`ui/login.py` dựng form đăng nhập / đăng ký (customer) trong một `Notebook`, kèm panel “LAN CSR mode” (Offline / máy Admin nhận CSR / máy Client gửi CSR). Hàm `on_login` gọi `auth.login`, nếu lỗi thì ghi audit `LOGIN_FAILED` và xóa ô mật khẩu.

Hai widget tái dụng giảm trùng lặp đáng kể:

- `ui/widgets/modal.py`: `init_modal` gom boilerplate tạo dialog (title, geometry, transient, grab_set); `fit_to_content` ép cửa sổ lớn lên nếu nội dung tràn (tránh che mất nút bấm); `make_button_row` render hàng nút chuẩn.
- `ui/widgets/status_table.py`: `StatusFilterTreeFrame` là bảng Treeview có toolbar lọc theo status + tô màu theo status + đếm số dòng, dùng lại cho hàng loạt view nghiệp vụ.

```

1 def init_modal(toplevel, *, parent, title, geometry="440x300",
2               resizable=False, padding=16) -> ttk.Frame:
3     toplevel.title(title)
4     if geometry: toplevel.geometry(geometry)
5     toplevel.resizable(resizable, resizable)
6     toplevel.transient(parent)
7     toplevel.grab_set()           # modal: khóa tương tác cửa sổ cha
8     body = ttk.Frame(toplevel, padding=padding)
9     body.pack(side=tk.TOP, fill=tk.BOTH, expand=True)
10    return body

```

3.12 Sao lưu dữ liệu — `scripts/backup_db.py`

Tiện ích sao lưu đáp ứng yêu cầu nộp D.2. Điểm kỹ thuật quan trọng: dùng **SQLite backup API** để chụp ảnh DB *nhất quán* kể cả khi app đang ghi, và sinh **manifest** kèm băm SHA-256 để kiểm tra toàn vẹn.

```

1 # Dùng SQLite backup API để snapshot consistent (kể cả khi DB đang được ghi)
2 src_conn = sqlite3.connect(str(db_path))
3 dst_conn = sqlite3.connect(str(db_out))
4 try:
5     src_conn.backup(dst_conn)
6 finally:
7     dst_conn.close(); src_conn.close()
8 manifest["files"].append({
9     "name": "ca_app.db", "size": db_out.stat().st_size,

```

```

10     "sha256": _sha256_file(db_out),
11 }
12 manifest["table_counts"] = _table_counts(db_out) # đếm row mỗi bảng để verify

```

Giải thích.

- `src_conn.backup(dst_conn)` sao chép trang DB theo cơ chế online backup của SQLite — an toàn hơn copy file thô.
- Manifest ghi *kích thước + SHA-256* từng file và *số dòng* mỗi bảng, để khi khôi phục có thể đối chiếu toàn vẹn.
- `master.key` *cực kỳ nhạy cảm*: backup có cờ `--no-master-key` để bỏ nó ra khi cần tránh rò rỉ; manifest ghi rõ cảnh báo. Mất cặp `master.key` + `ca_app.db` là không decrypt được private key.

4 Cần nắm để vấn đáp

Hỏi: scrypt và AES-GCM khác nhau ở đâu, vì sao chọn từng loại cho từng việc?

Đáp: scrypt là hàm *băm/dẫn xuất khóa một chiều* dùng cho mật khẩu — ta chỉ cần *kiểm chứng* mật khẩu chứ không cần lấy lại, nên một chiều là phù hợp và an toàn nhất. AES-256-GCM là *mã hóa đối xứng hai chiều* dùng cho private key — vì ta *cần dùng lại* khóa đó để ký, nên phải giải mã được. Hai nhu cầu khác nhau $\Rightarrow$ hai công cụ khác nhau.

Hỏi: Vì sao mật khẩu băm bằng scrypt thì không thể giải ngược?

Đáp: scrypt là hàm một chiều: tính hash từ mật khẩu thì dễ, nhưng đảo ngược từ hash về mật khẩu là không khả thi về tính toán (phải dò vét toàn bộ không gian mật khẩu). Cộng thêm salt ngẫu nhiên cho mỗi mật khẩu và chi phí bộ nhớ cao (memory-hard) khiến tấn công bằng rainbow table hay brute-force offline trở nên rất tốn kém.

Hỏi: `master.key`, nonce và AAD trong AES-GCM dùng để làm gì?

Đáp: `master.key` là khóa 256-bit dùng chung để mã/giải mã. nonce là số dùng-một-lần 12 byte, mới mỗi lần mã hóa, tuyệt đối không tái dùng với cùng khóa (lặp nonce phá vỡ tính an toàn của GCM); nonce được lưu kèm ciphertext. AAD là dữ liệu phụ *không mã hóa nhưng được xác thực*, dùng để cột ciphertext vào ngữ cảnh (`table:row_id`) — nếu ai đó hoán đổi bản ghi thì AAD không khớp và giải mã thất bại (`InvalidTag`).

Hỏi: PRAGMA foreign_keys và WAL để làm gì?

Đáp: SQLite mặc định *tắt* kiểm tra khóa ngoại, nên phải bật `foreign_keys = ON` cho mỗi connection thì các ràng buộc REFERENCES (cascade/restrict/set null) mới có hiệu lực — đảm bảo toàn vẹn dữ liệu. WAL (Write-Ahead Logging) là chế độ ghi nhật ký trước, cho phép *đọc và ghi đồng thời* mượt hơn — phù hợp app có UI và server nên cùng truy cập DB.

Hỏi: Tại sao login vẫn gọi `verify_password` khi username không tồn tại?

Đáp: Để chống dò tài khoản (user-enumeration) qua kênh thời gian. Nếu username sai mà trả lời ngay lập tức, còn username đúng thì mất $\approx 100\text{ms}$ để verify, kẻ tấn công đo thời gian sẽ biết username nào có thật. Bằng cách verify trên một hash giả khi không tìm thấy user, thời gian phản hồi hai trường hợp gần như bằng nhau. Thông báo lỗi cũng để chung “Sai username hoặc password”.

Hỏi: App định tuyến theo role như thế nào?

Đáp: Sau khi `auth.login` trả về dict có `role`, hàm `App.on_login_success` lưu `session` rồi kiểm tra: `role "admin" → show_admin_dashboard`, `role "customer" → show_customer_dashboard`. Đây là single Tk root + swap content, mỗi dashboard có sidebar menu riêng phù hợp quyền hạn.

Hỏi: Làm sao decode và đọc các trường của một certificate?

Đáp: Dùng `x509.load_pem_x509_certificate` của thư viện `cryptography` để parse PEM thành đối tượng cert, rồi truy cập các thuộc tính: `version`, `serial_number`, `signature_algorithm_oid`, `issuer/subject (rfc4514_string())`, `not_valid_before_utc/not_valid_after_utc`, `public_key()` (loại + `key_size`), và lặp `cert.extensions` để liệt kê các phần mở rộng kèm cờ `critical`. Toàn bộ bọc `try/except` để PEM hỏng không làm sập UI.

Hỏi: Audit log được thiết kế thế nào để không phá nghiệp vụ?

Đáp: `write_audit` là best-effort: toàn bộ thao tác ghi DB bọc trong `try/except`, nếu lỗi chỉ in ra `stderr` chứ không ném exception. Nhờ đó một sự cố khi ghi log (vd DB khóa) không làm hỏng thao tác chính như đăng nhập hay phát hành cert. Tên action lấy từ catalog `Action` để nhất quán.

Hỏi: Vì sao `transaction` cần ROLLBACK còn `conn_scope` thì không?

Đáp: `transaction` dùng cho ghi nên phải đảm bảo nguyên tử: nếu giữa chừng có lỗi thì ROLLBACK để DB không rơi vào trạng thái nửa vời. `conn_scope` chỉ đọc, không thay đổi dữ liệu nên không cần BEGIN/COMMIT/ROLLBACK — chỉ cần mở và đóng connection cho gọn.

5 Test case & demo

Cụm này có bộ smoke test sẵn: `tests/test_m1_foundation.py`. Lệnh chạy:

```
1 # từ thư mục gốc dự án
2 python tests/test_m1_foundation.py
```

5.1 Bảng test case chính

Test	Kiểm chứng điều gì
<code>test_db_init_creates_tables</code>	<code>init_db</code> tạo đủ 9 bảng.
<code>test_password_hash_roundtrip</code>	Hash bắt đầu <code>scrypt\$</code> ; verify đúng/sai; hai lần hash cùng mật khẩu phải khác (salt random).
<code>test_aes_gcm_roundtrip</code>	Mã/giải mã đúng; nonce 12 byte; AAD sai → <code>InvalidTag</code> ; sửa ciphertext → <code>InvalidTag</code> .
<code>test_register_login_flow</code>	Đăng ký, trùng username báo lỗi, login đúng/sai, <code>last_login_at</code> được cập nhật.
<code>test_change_password</code>	Sai mật khẩu cũ báo lỗi; mới = cũ báo lỗi; đổi OK thì login bằng mật khẩu cũ thất bại, mật khẩu mới thành công.
<code>test_seed_admin_if_empty</code>	Lần đầu tạo admin; lần sau idempotent (trả None).
<code>test_audit_log</code>	Ghi 3 event, thứ tự mới nhất trước, JSON details, filter theo action.
<code>test_system_config</code>	Seed defaults + get/set + whitelist guard.

5.2 Đoạn unit test gợi ý (tự kiểm chứng nhanh khi demo)

```
1 # Băm + verify mật khẩu
2 h = hash_password("SuperSecret#123")
3 assert h.startswith("scrypt$")
4 assert verify_password("SuperSecret#123", h)
5 assert not verify_password("wrong", h)
6 assert hash_password("x") != hash_password("x") # salt ngẫu nhiên
7
8 # AES-GCM: roundtrip + phát hiện chỉnh sửa
9 nonce, ct = encrypt_blob(b"PRIVATE-KEY", aad=b"customer_keys:42")
10 assert decrypt_blob(nonce, ct, aad=b"customer_keys:42") == b"PRIVATE-KEY"
11 from cryptography.exceptions import InvalidTag
12 try:
```

```
13     decrypt_blob(nonce, ct, aad=b"customer_keys:43")    # AAD sai
14     assert False
15 except InvalidTag:
16     pass
```

5.3 Kịch bản demo trực quan

1. **Khởi động:** `python main.py`. Lần đầu hệ thống tạo `ca_app.db`, `master.key`, seed admin (`admin/Admin@123`).
2. **Đăng nhập admin** → vào Admin Dashboard. **Đăng xuất, đăng ký** một customer mới → tự động vào Customer Dashboard (minh họa định tuyến theo role).
3. **Đổi mật khẩu:** mở `ChangePasswordDialog`, thử sai mật khẩu cũ (báo lỗi), rồi đổi đúng.
4. **Mở chi tiết cert** (tab **Decoded**): cho thấy Subject/Issuer/ Serial/PublicKey/SigAlgo/validity/extension được decode từ PEM.
5. **Backup DB:** chạy `python scripts/backup_db.py` → xem thư mục `backups/<timestamp>/` và `backup_manifest.json` với SHA-256 + table counts.

6 Tích hợp vào chương trình chung

Cụm nền tảng được các module nghiệp vụ khác “ghép” vào theo các điểm nối sau:

Điểm nổi	Được dùng bởi
<code>get_conn / transaction / conn_scope</code>	Tất cả service đọc/ghi DB (auth, system_config, ca, csr, cert_lifecycle, revocation, audit...).
<code>encrypt_blob / decrypt_blob</code>	Module sinh keypair và Root CA để cất/đọc private key đã mã hóa.
<code>hash_password / verify_password</code>	<code>auth.py</code> cho login, đăng ký, đổi mật khẩu.
<code>write_audit + Action</code>	Mọi flow cần ghi nhật ký (login, duyệt CSR, phát hành/thu hồi cert, publish CRL).
<code>App (router) + apply_theme</code>	Mọi dashboard/view nghiệp vụ render bên trong khung này và dùng style chung.
<code>CertDetailDialog</code>	Admin (xem cert đã phát hành) và customer (xem cert của mình) đều mở dialog này.
<code>StatusFilterTreeFrame + init_modal</code>	Các view CSR queue, cert management, my certs...tái dùng để dựng bảng + dialog.
<code>schema.sql</code> (9 bảng)	Toàn bộ dữ liệu của hệ thống nằm trên lược đồ này.

Thứ tự khởi tạo bắt buộc (Huy cần nắm để giải thích lỗi nếu có): trong `App._init__`, phải `_bootstrap()` (init DB + seed) *trước*, rồi mới tạo `tk.Tk()` và `apply_theme(root)` (vì style cần root), cuối cùng `show_login()`. Đảo thứ tự sẽ gây lỗi “no Tk root” hoặc đăng nhập vào DB chưa có bảng.

7 Bổ sung sau rà soát: các feature dễ bị hỏi

7.1 Wiring chế độ LAN trong `app.py` & `login.py`

Khung GUI là nơi *ghép* chế độ LAN (phần nghiệp vụ thuộc cụm Quân), nên Huy cần nắm phần wiring:

- `App.start_csr_api / stop_csr_api / set_remote_csr_api`: bật/tắt server nhận CSR trên máy Admin, hoặc trở client tới Admin API. Đọc 3 biến môi trường `X509_REMOTE_CSR_API_URL`, `X509_CSR_API_TOKEN`, `remote_csr_password`.
- Panel LAN trong `LoginFrame` (chiếm phần lớn `login.py`): 3 chế độ *Offline / Admin* (nhận CSR) / *Client* (gửi CSR). Ràng buộc: `on_start_admin_api` validate port 1024–65535 và *bắt buộc token* khi bind ra LAN; `on_test_client_api` gọi `check_admin_api_health`.
- `_sync_lan_crl_ocsp_urls`: ghi lại `prod_crl_url/prod_ocsp_url` theo *IP LAN* (qua `set_`

`config`) để URL nhúng trong cert (CRLDP/AIA) truy cập được từ máy khác — nếu không, máy client verify sẽ không tải nổi CRL/OCSP.

7.2 Quy tắc hợp lệ username & mật khẩu (`auth.py`)

- `_validate_username`: username chỉ gồm chữ-số và `.` `_` `-` `@`, tối đa **64** ký tự (chống ký tự lạ/đụng định dạng).
- Mật khẩu tối thiểu **6** ký tự (`MIN_PASSWORD_LEN`); `role` phải thuộc whitelist (`'admin'`, `'customer'`).
- `get_user_by_id` trả profile *không* kèm `password_hash`; `count_users(role)` là điều kiện idempotent của `seed_admin_if_empty` (chỉ tạo admin khi chưa có admin nào).

7.3 Server hạ tầng nền & Backup/Restore đầy đủ

- **Prod servers.** `_bootstrap` gọi `infra_manager.start_prod_servers()` bật *nền* 2 HTTP server: CRL :8889 + OCSP :8888 (phục vụ verify cert, vd upload-verify B.9); `_on_close` gọi `stop_all()` khi đóng app; admin dashboard có status bar badge ON/OFF theo port.
- **Backup** (`scripts/backup_db.py`) sao lưu **nhiều hơn DB**: ngoài `ca_app.db` (qua SQLite backup API), còn copy `master.key` (kèm cảnh báo), `shutil.copypath` toàn bộ `certs/` (`trust_store` + `crl.pem` + `ocsp_db.json`, băm SHA-256 từng file), và `schema.sql`; tất cả ghi vào `manifest`. Cờ CLI: `--dest`, `--no-master-key`, `--dump-sql` (và `--schema-only` ghi trong docstring nhưng *chưa* khai báo trong `argparse` — điểm dễ bị hỏi).
- `restore_hint()` in quy trình khôi phục: đóng app → backup trạng thái hiện tại → copy db+master.key+certs → khởi động lại → đối chiếu số row theo manifest. *Cảnh báo*: `master.key` và `ca_app.db` phải khớp *cặp* — lệch cặp ⇒ `InvalidTag` khi giải mã private key.

7.4 Chi tiết nhỏ khác

- **Font**: chain hiện chỉ có `['Montserrat']`; `resolve_family` duyệt chain và fallback phần tử cuối (vẫn ra Montserrat kể cả khi máy chưa cài). Lưu ý docstring `theme.py` còn tàn dư “DFVN Float” — thực thi dùng Montserrat; nói rõ để không lúng túng nếu giám khảo đọc code.
- `reset_master_key_cache()` xóa cache `_master_key_cache` (dùng cho test đổi `DEFAULT_MASTER_KEY_PATH`); `_load_or_create` kiểm tra key đúng **32 byte** và `chmod 0o600` (POSIX).
- `hero_section` (hero + lưới feature card, dùng ở overview & Lab) và `build_dashboard_header` (badge role + nút Đổi mật khẩu mở `ChangePasswordDialog` + Đăng xuất gọi `app.logout`) là component UI tái dụng.

8 Checklist chuẩn bị vấn đáp

- ☐ Giải thích được `main.py` neo cwd để làm gì và hậu quả nếu không neo.
- ☐ Chỉ ra trong `connection.py` ba dòng PRAGMA/cấu hình và nói rõ tác dụng từng dòng (`foreign_keys`, `row_factory`, `WAL`).
- ☐ Phân biệt `transaction` (ghi, có `BEGIN/COMMIT/ROLLBACK`) và `conn_scope` (đọc).
- ☐ Vẽ/đọc được lược đồ 9 bảng và giải thích 3 chính sách khóa ngoại (`CASCADE` / `RESTRICT` / `SET NULL`).
- ☐ So sánh `scrypt` vs `AES-GCM`: một chiều vs hai chiều, vì sao chọn từng loại.
- ☐ Giải thích vì sao hash mật khẩu không giải ngược được và vai trò của `salt`.
- ☐ Nói rõ vai trò `master.key`, `nonce` (không tái dùng) và `AAD` trong `AES-GCM`; điều gì xảy ra khi `ciphertext` bị sửa.
- ☐ Trình bày cơ chế chống dò tài khoản trong `login` (`dummy verify` + `message chung`).
- ☐ Giải thích định tuyến theo `role` trong `on_login_success`.
- ☐ Demo `CertDetailDialog` và đọc được ý nghĩa từng trường X.509 (`Subject/Issuer/Serial/PublicKey/Sig`).
- ☐ Giải thích `audit log best-effort` và catalog `Action`.
- ☐ Mô tả backup: `SQLite backup API` + `manifest SHA-256` + cảnh báo `master.key`.
- ☐ Chạy được `tests/test_m1_foundation.py` và đọc kết quả `PASS/FAIL`.
- ☐ Nắm thứ tự `bootstrap` trong `App` và lý do bắt buộc của thứ tự đó.